WEEK 1 · DAY 1 · OVERVIEW

Tree Instability and the Bootstrap

From your first line of code to your own AI agents.
Every lesson is a fixed income problem.



Created by **Jeff Lenamon**

ABOUT 45 MIN WITH EXERCISES / 11 SECTIONS / 4 EXERCISES

This course is educational content created in a personal capacity. Nothing here is investment advice or a recommendation to buy or sell any security. All portfolio data is synthetic.

Problem Statement

BUSINESS CONTEXT

Course 5 combines many models into one: bagging, random forests, boosting and stacking, then reads what the combined model leans on.

It starts where Course 4 ended, with the card file's 18,000 training and 6,000 validation accounts, trees, pruning, cross-validation and ROC AUC.

Course 4's Day 9 ended with a 12-rule pruned tree a credit committee could read, and a reviewer asks one question: had the bank kept a different 18,000 accounts, would a tree say the same thing about this account?

The bootstrap draws other plausible training sets from the one we have, so we can answer by experiment.

The accounts are real, from Taiwan in 2005, so they describe consumer credit there and then, not US credit today.

The four person columns are never features, because a model that used them would sort people by them.

OBJECTIVES

Rebuild Course 4's rows and its two trees, draw a bootstrap sample and count the accounts it leaves out, and fit a tree on each of ten samples.

Measure how far one account's output moves from tree to tree, compare deep and shallow trees, average the trees and score the average on validation rows, and name the rows each tree never saw.

ABOUT 45 MIN WITH EXERCISES / 11 SECTIONS / 4 EXERCISES

Your Route Through Day 1

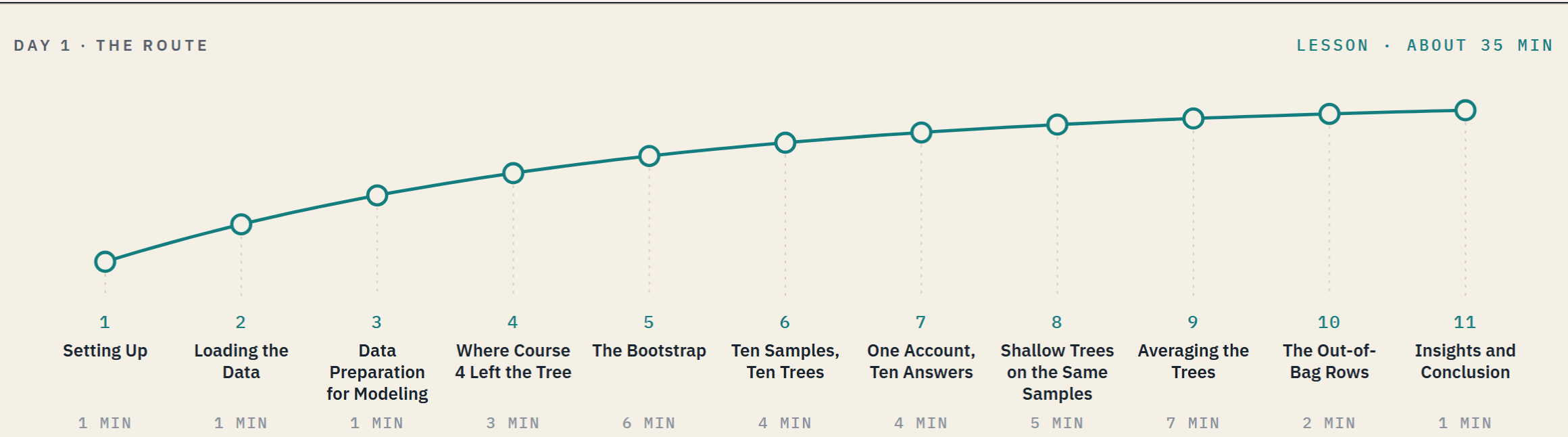


FIGURE 1 Eleven sections, in order, with about how long each takes.

Time. About 35 minutes for the lesson, then about 12 for the four exercises.

Fix the seed and load the libraries.

Setting Up

THE SETUP

```
import os
from pathlib import Path
os.environ["OMP_NUM_THREADS"] = "1"
os.environ["OPENBLAS_NUM_THREADS"] = "1"
SEED = 42
DATA = "../../../data/"
if not Path(DATA + "benchmark.csv").exists():
    DATA = "https://raw.githubusercontent.com/lenamonj/python-fixed-income/main/data/"
```

THE LIBRARIES

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.tree import DecisionTreeClassifier, plot_tree
from sklearn.metrics import roc_auc_score, average_precision_score, roc_curve
```

Read the card file and pick 19 columns.

Loading the Data

THE FILE'S COLUMNS

- **LIMIT_BAL**: credit limit, NT dollars
- **PAY_0**, **PAY_2** to **PAY_6**: repayment status, September back to April 2005, where -1 is paid on time, 1 to 9 is months late, and -2 and 0 are undocumented (used as numbers, a modelling choice, as in Course 4)
- **BILL_AMT1** to **BILL_AMT6**: the bill, NT dollars, September back to April
- **PAY_AMT1** to **PAY_AMT6**: the amount paid, NT dollars, September back to April
- **default payment next month**: 1 if the account defaulted the following month, **the target**
- **ID**, **SEX**, **EDUCATION**, **MARRIAGE**, **AGE**: never used

READING THE CARD FILE

```
card = pd.read_csv(DATA + "credit_card_default.csv")
TARGET = "default payment next month"
print(len(FEATURES), "features")
```

```
19 features
```

THE SHAPE AND THE DEFAULT SHARE

```
print(card.shape)
```

```
(30000, 25)
```

Rebuild Course 4's training, validation and test rows.

Data Preparation for Modeling

COURSE 4'S SPLIT

```
X_rest, X_test, y_rest, y_test = train_test_split(X, y, test_size=0.2, stratify=y, random_state=SEED)
X_train, X_valid, y_train, y_valid = train_test_split(X_rest, y_rest, test_size=0.25, stratify=y_rest, random_state=SEED)
print("Training:", len(X_train), " Validation:", len(X_valid), " Test:", len(X_test))
print("Default share:", round(y_train.mean(), 4), round(y_valid.mean(), 4), round(y_test.mean(), 4))
```

```
Training: 18000  Validation: 6000  Test: 6000
Default share: 0.2212 0.2212 0.2212
```

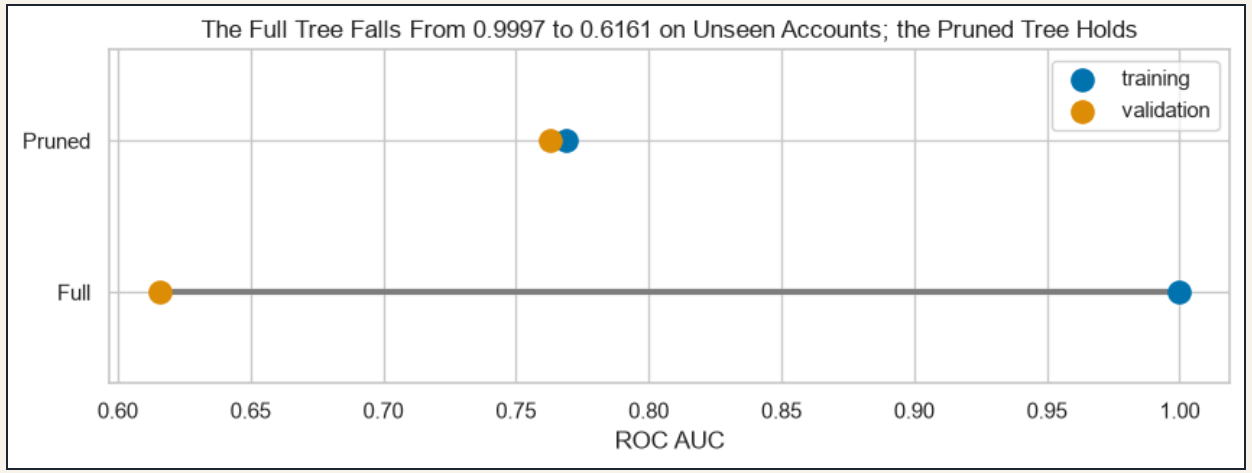
WHO DOES WHAT

The same 18,000, 6,000 and 6,000 accounts as Course 4, each with the same default share.

Every score in this notebook is on the validation rows.

Training rows fit, validation rows report, test rows stay closed.

Where Course 4 Left the Tree



THE TWO TREES

	LEAVES	TRAINING	VALIDATION
Full	2750	0.9997	0.6161
Pruned	12	0.7691	0.7627

OUTPUTS OF THE FULL TREE

```
full_outputs = full_tree.predict_proba(X_valid)[:, 1]
print("Outputs at exactly 0 or 1:", round(((full_outputs == 0) | (full_outputs == 1)).mean() * 100, 1), "percent")
```

```
Outputs at exactly 0 or 1: 97.7 percent
```

Course 4 asked how much a tree memorises. Today asks how much of a tree is the luck of its 18,000 rows. To see that, we need other sets of 18,000.

Draw 18,000 rows with replacement, then count repeats.

The Bootstrap

A BOOTSTRAP SAMPLE

Course 4's Day 3 drew 615 bonds with replacement, `rng.choice(615, size=615)`, and refitted a regression on each draw.

Today the same draw runs on the 18,000 training accounts.

ONE DRAW OF 18,000 POSITIONS

```
rng = np.random.default_rng(SEED)
n = len(X_train)
picks = rng.choice(n, size=n)
distinct = len(np.unique(picks))
```

COUNTING HOW OFTEN EACH ACCOUNT WAS DRAWN

```
times = np.bincount(picks, minlength=n)
drawn_counts = pd.Series(times).value_counts().sort_index()
drawn_counts
```

The sample still holds 18,000 rows: the repeats fill the places of the accounts left out.

Ten Samples, Ten Trees

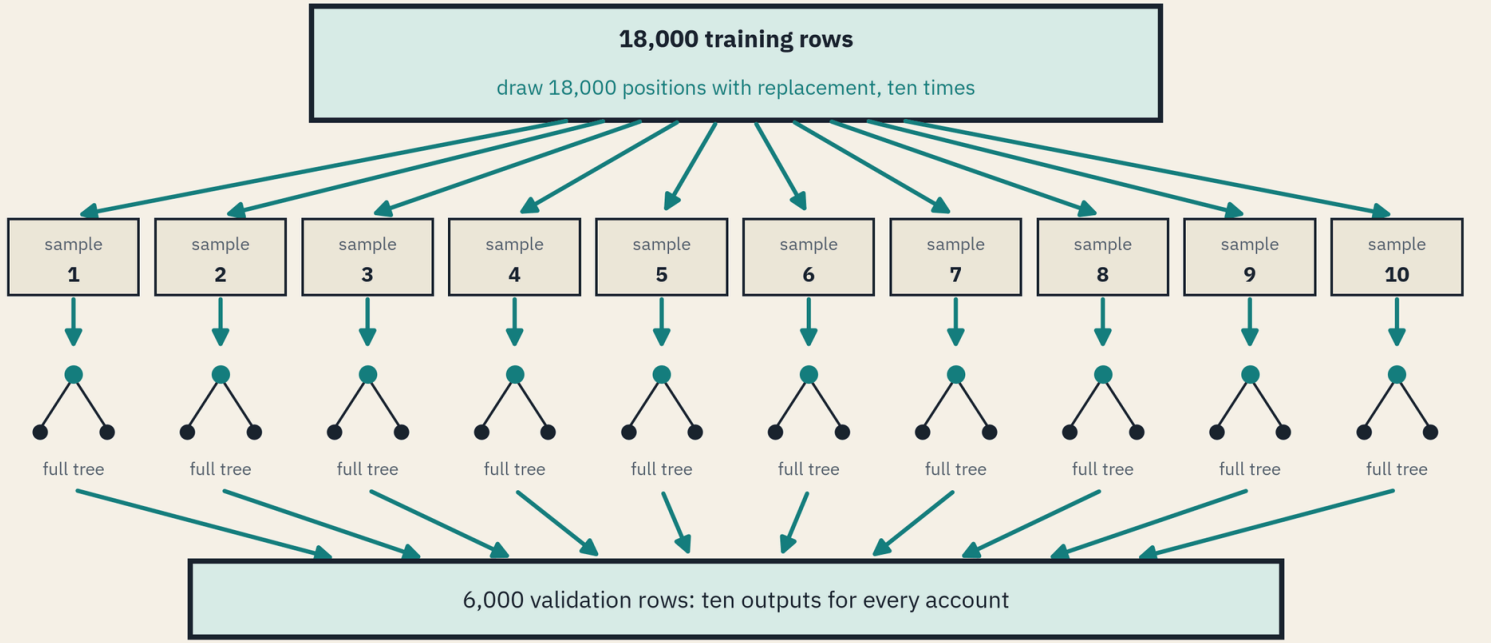


FIGURE 2 Ten bootstrap samples of the 18,000 training rows, a full tree on each, all scored on the same 6,000 validation rows.

WHAT THE TEN TREES ASK

	ROOT	LEAVES	DEPTH
1	PAY_0 <= 1.5	1877	43
2	PAY_0 <= 1.5	1878	35
3	PAY_0 <= 1.5	1902	37
4	PAY_0 <= 1.5	1905	34
5	PAY_0 <= 1.5	1856	35

The ten trees agree at the top and score alike. Next we ask whether they agree about any one account.

One Account, Ten Answers

TEN OUTPUTS FOR EACH ACCOUNT

	TREE 1	TREE 2	TREE 3	TREE 4	TREE 5
0	0.0000	1.0000	0.0000	0.0000	0.0000
1	0.0000	0.0000	0.0000	0.0000	1.0000
2	0.5714	0.7000	0.7000	0.5714	0.3000
3	0.0000	0.0000	0.0000	0.0000	0.0000
4	1.0000	0.0000	0.0000	1.0000	0.0000

A VOTE

```
votes = (outputs >= 0.5).sum(axis=1)
```

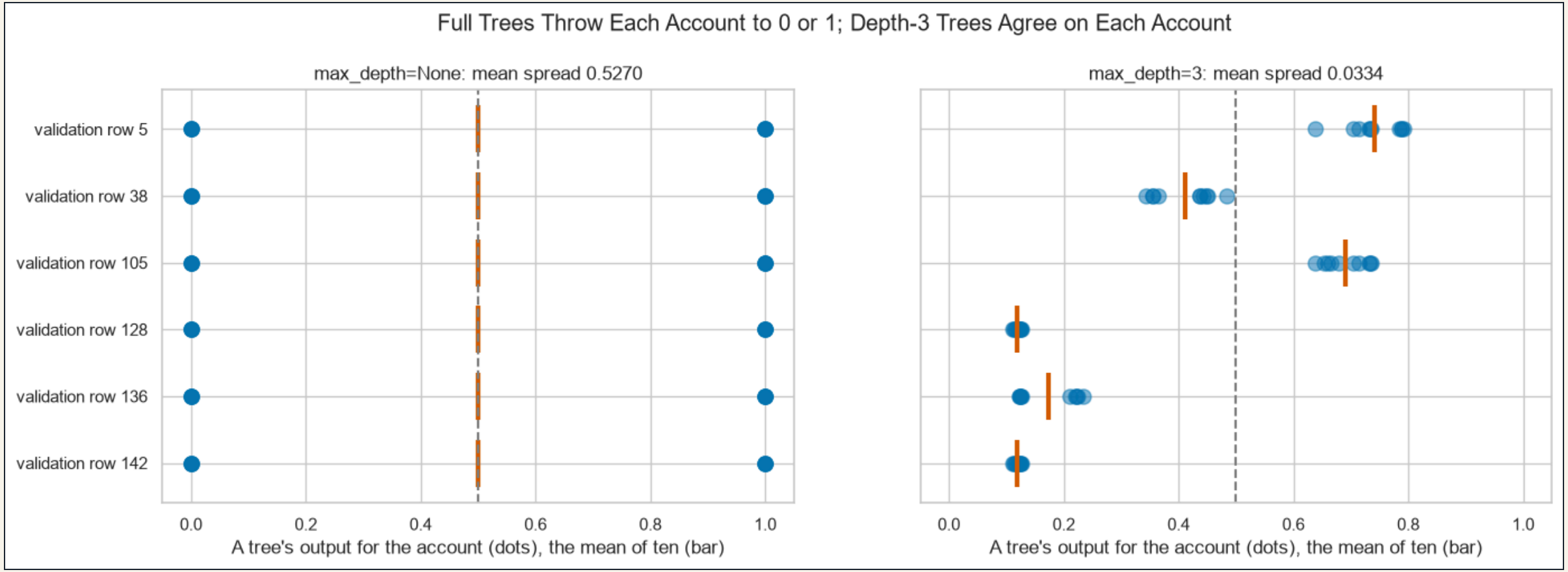
VARIANCE

Each tree is almost always certain about an account, and the ten are certain in different ways.

Variance is how far a model's output for the same account moves when the model is fitted on another sample.

The votes still sort the accounts: their share is a steadier number than any one tree's flag.

Shallow Trees on the Same Samples



Deep trees are noisy and shallow trees are limited. Averaging works on noise, so we average both kinds and compare.

Averaging the Trees

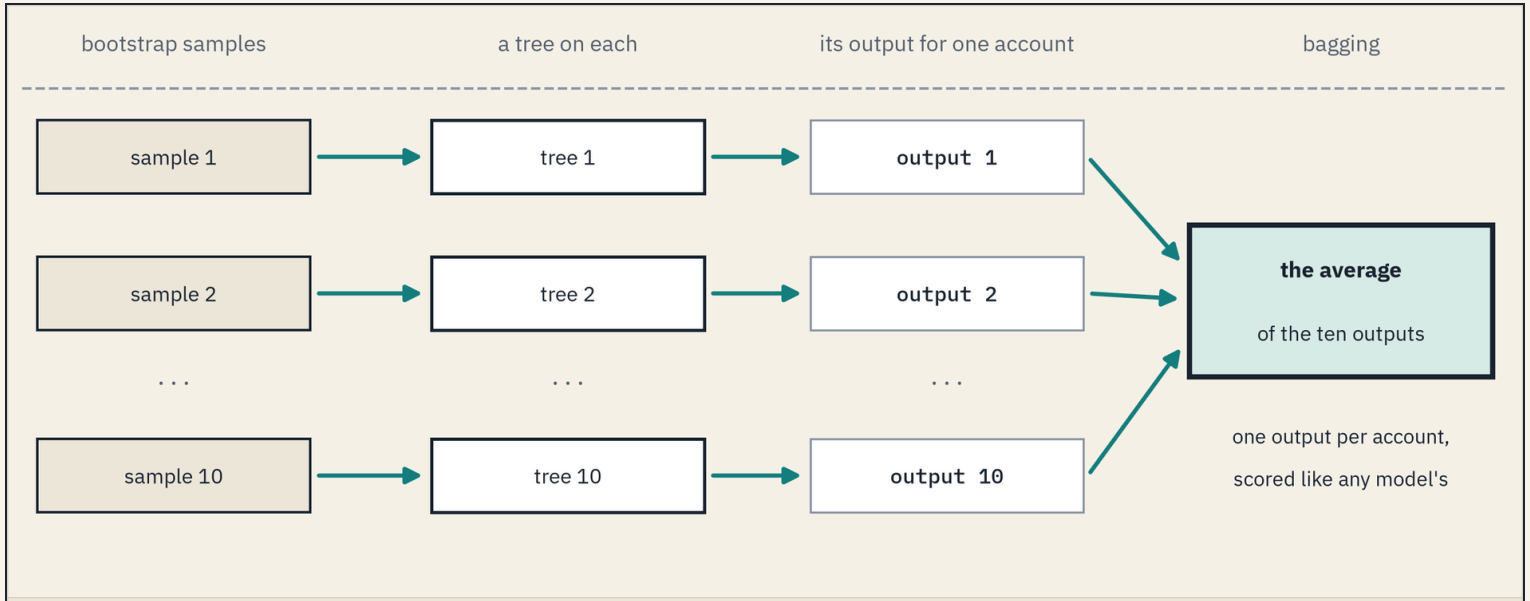


FIGURE 3 Bagging: a tree on each bootstrap sample, its output for an account, and the ten outputs averaged into one.

BAGGING

An **ensemble** is a model made of several models whose outputs are combined.

Bagging, short for bootstrap aggregating, fits a model on each bootstrap sample and averages their outputs.

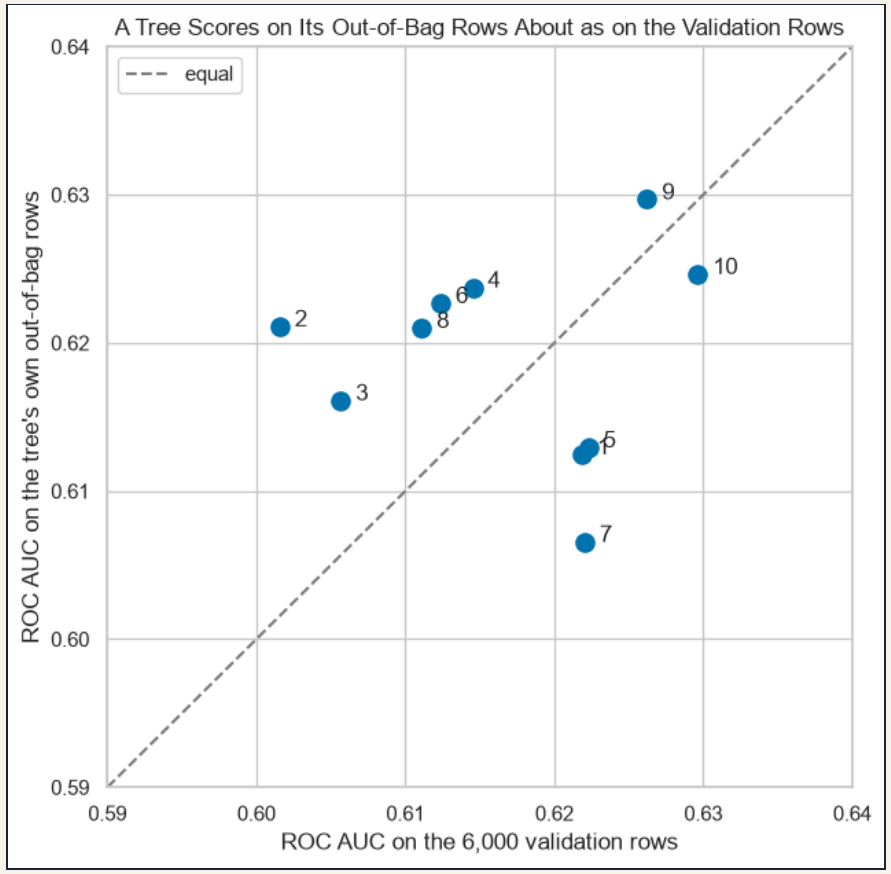
THE FIRST LINE OF CODE

```
average = outputs.mean(axis=1)
```

Bagging, short for bootstrap aggregating, fits a model on each bootstrap sample and averages their outputs.

Score each tree on rows it never drew.

The Out-of-Bag Rows



ROWS A TREE NEVER SAW

```
rows = []
for sample, tree in zip(samples, full_trees):
    unseen = np.bincount(sample, minlength=n) == 0
    rows.append([unseen.sum(), auc(tree, X_train[unseen], y_train[unseen])])
```

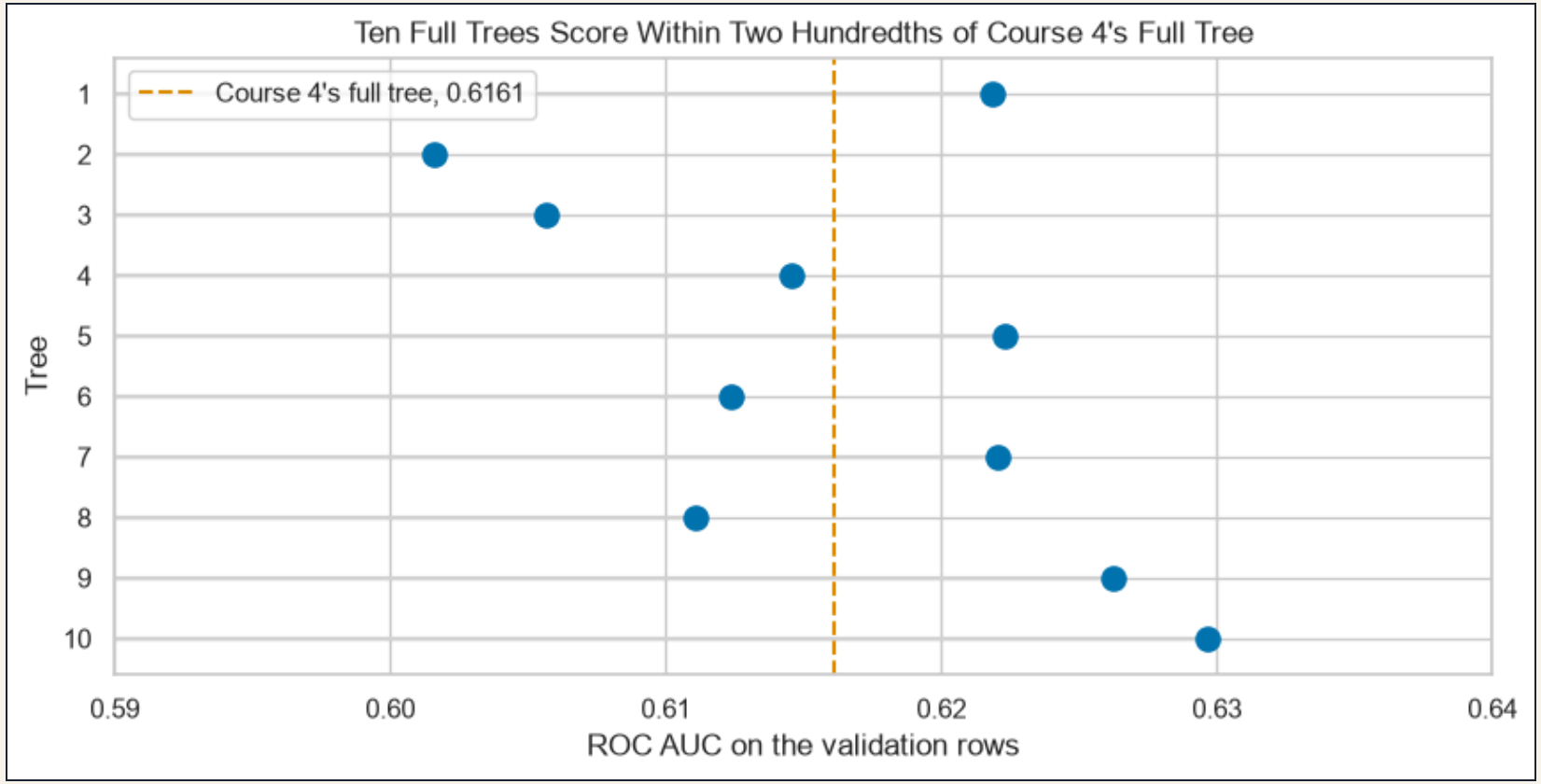
WHAT THE CHART SHOWS

Its score on them lands within two hundredths of its score on the validation rows.

The rows a tree never saw are a free validation set for that tree.

Every bootstrap tree carries its own unseen rows; Day 2 turns them into one score for the whole average.

Insights and Conclusion



THE TEN TREES' SCORES

The ten trees score 0.6016 to 0.6296, around Course 4's full tree at 0.6161.

One tree is one draw of luck; Day 2 averages hundreds.

You Can Now

- Draw a bootstrap sample with `rng.choice` and count what it leaves out.
- Say why the share left out settles near a fixed number as the file grows.
- Fit a tree on each of several samples and read their top questions with `.tree_`.
- Measure how far one account's output moves from tree to tree, and name variance and bias.
- Average trees, score the average, and find each tree's out-of-bag rows.

NEXT

WEEK 1 · DAY 2

Bagging and Out-of-Bag Scores

Day 2 grows the average to hundreds of trees and scores it on unseen rows.